

# Introduction to Compilers

## Problem Set: Compilers & Computational Theory

**Instructions.** This set has six exercises tied directly to the lecture. Exercises 1 and 4 are self-checking (a written solution is provided separately — use it to verify your own work). Exercises 5 and 6 will be discussed together in class before you finalize your answers. Show your work throughout; for proofs, follow the reduction template used in lecture.

---

### 1 Lexical Analysis: Building a DFA [Self-check — solution provided]

Recall the 3-state DFA from lecture that recognizes the keyword `if` (accepting on input `i`, `f`).

- (a) Design a DFA that recognizes the keyword `while`. Give both a state diagram and the formal 5-tuple  $(Q, \Sigma, \delta, q_0, F)$ .
- (b) Design a second DFA (or extend the first) that recognizes *identifiers*: a letter followed by zero or more letters or digits. Give a regular expression for this language as well.
- (c) Using the *closure under union* property discussed in lecture, explain **in your own words** how a single lexer could combine your two DFAs above to recognize both keywords and identifiers in a single left-to-right pass over the input, without ever confusing one for the other.

### 2 Syntax Analysis: A New Grammar

Consider the following context-free grammar, extending the lecture's grammar to handle `while` loops:

$$\begin{aligned} S &\rightarrow \texttt{while} ( E ) S \\ S &\rightarrow \texttt{id} = E \\ E &\rightarrow \texttt{id} \mid \texttt{num} \mid E < E \end{aligned}$$

- (a) Using this grammar, draw the parser's stack at four checkpoints (as in lecture) while matching the input `while ( i < n ) sum = i`.
- (b) At the checkpoint immediately after the literals `while` and `(` have been matched, what symbol is on top of the stack? Explain why `while` and `(` themselves never appear as a lingering stack entry in your diagram.

- (c) This grammar, by itself, cannot guarantee that `i` and `n` were declared before this statement runs. Which compiler phase is responsible for catching that, and why can't the grammar do it instead?

### 3 Semantic Analysis: Beyond Context-Free

- (a) In Computational Theory, you showed that  $\{a^n b^n c^n \mid n \geq 0\}$  is not context-free. In 4–6 sentences, explain the structural similarity between that result and the claim that “every variable must be declared before its first use” cannot be enforced by any context-free grammar.
- (b) In your own words, describe how a real compiler enforces the declare-before-use rule anyway, despite part (a).
- (c) A classmate suggests: “just use a context-sensitive grammar (recognized by a linear-bounded automaton) instead of a CFG, so parsing enforces declare-before-use directly.” This is theoretically possible. Explain, using the complexity class mentioned in lecture, why it's a poor engineering choice for a real compiler.

### 4 Optimization: Reading a Real Compiler [Self-check — solution provided]

For this exercise you will use Compiler Explorer ([godbolt.org](http://godbolt.org)).

- (a) Write a short C function equivalent in structure to `findIndex` from lecture (a loop with an early return). Compile it with `x86-64 gcc` at optimization levels `-O0` and `-O2`. Paste both assembly outputs.
- (b) Identify at least two concrete differences between the two outputs that match a pattern from lecture (e.g. register allocation, a branch-free rewrite using a conditional move, instruction fusion).
- (c) Count the number of stack memory accesses (instructions referencing `[rbp-...]` or similar) in each version. What does the difference tell you about the cost model discussed in lecture?

### 5 Decidability: A Reduction Proof [We'll go over this together]

Define the language

$\text{ALWAYS-HALTS-OPT} = \{\langle O \rangle \mid O \text{ is a program implementing a compiler optimization pass, and } O \text{ halts on every input}\}$

- (a) State precisely what it would mean for ALWAYS-HALTS-OPT to be decidable.

- (b) Prove that ALWAYS-HALTS-OPT is undecidable, by reduction from the Halting Problem  $HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$ . Follow the reduction template from lecture: given  $\langle M, w \rangle$ , construct a program  $O_{M,w}$  such that  $O_{M,w} \in \text{ALWAYS-HALTS-OPT}$  if and only if  $M$  halts on  $w$ .
- (c) In 2–3 sentences, explain how this result justifies the claim from lecture that real optimizers use sound, conservative approximations rather than trying to answer questions like this exactly.
- (d) (*Open-ended.*) Name one other compiler question — not discussed in lecture — that you believe reduces to the Halting Problem, and briefly sketch why.

## 6 Integrative: Positioning a New Phase [We’ll go over this together]

Many modern compilers (e.g. Rust) include a *borrow checker*: a compile-time phase that verifies memory-safety rules (such as “a value is never used after it’s been moved”) before code generation.

- (a) Where would you place borrow-checking in the nested hierarchy diagram from lecture — Type 3, Type 2, Type 1, or Type 0? Justify your placement by describing what kind of memory or context-tracking the check requires.
- (b) Is borrow-checking, as you’ve placed it, guaranteed to be decidable? Explain.
- (c) Propose one additional compiler phase or check not discussed in lecture, and repeat parts (a)–(b) for it.